

RAJALAKSHMI ENGINEERING COLLEGE

RAJALAKSHMI NAGAR, THANDALAM – 602 105



AI23331
FUNDAMENTALS OF MACHINE LEARNING
LABORATORY

Lab Manual

Name :

Year / Branch / Section :

Register No. :

Semester :

Academic Year :



RAJALAKSHMI ENGINEERING COLLEGE
An Autonomous Institution

BONAFIDE CERTIFICATE

Name:

Academic Year: **Semester:** **Branch:**

Register No.

*Certified that this is the bonafide record of work done by the above student in
the.....Laboratory
during the academic year 2025- 2026*

Signature of Faculty in-charge

Submitted for the Practical Examination held on.....

INDEX

Reg. No. : _____ Name : _____

Year : _____ Branch : _____ Sec : _____

S. No.	Date	Title	Page No.	Teacher's Signature / Remarks
1		A Python Program To Implement Univariate Regression, Bivariate Regression And Multivariate Regression		
2		A Python Program To Implement Simple Linear Regression Using Least Square Method		
3		A Python Program To Implement Logistic Model		
4		A Python Program To Implement Single Layer Perceptron		
5		A Python Program To Implement Multi Layer Perceptron With Back Propagation.		
6		A Python Program To Do Face Recognition Using Svm Classifier.		
7		A Python Program To Implement Decision Tree.		
8		A Python Program To Implement Boosting.		
9		A Python Program To Implement Knn And K-Means		

10		A Python Program To Implement Dimensionality Reduction – PCA		
----	--	--	--	--

Ex. No. : 01

Date:

Register No. : 231701021

Name: Isaitamil S

Univariate, Bivariate and Multivariate Analysis of Iris Dataset using Python

Aim:

To perform univariate, bivariate, and multivariate analysis on the Iris dataset using Python libraries.

Program:

```
#Import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
#Load Dataset path =
"/content/drive/MyDrive/MLDataset/Iris.csv" df
= pd.read_csv(path)
df.head()
```

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa

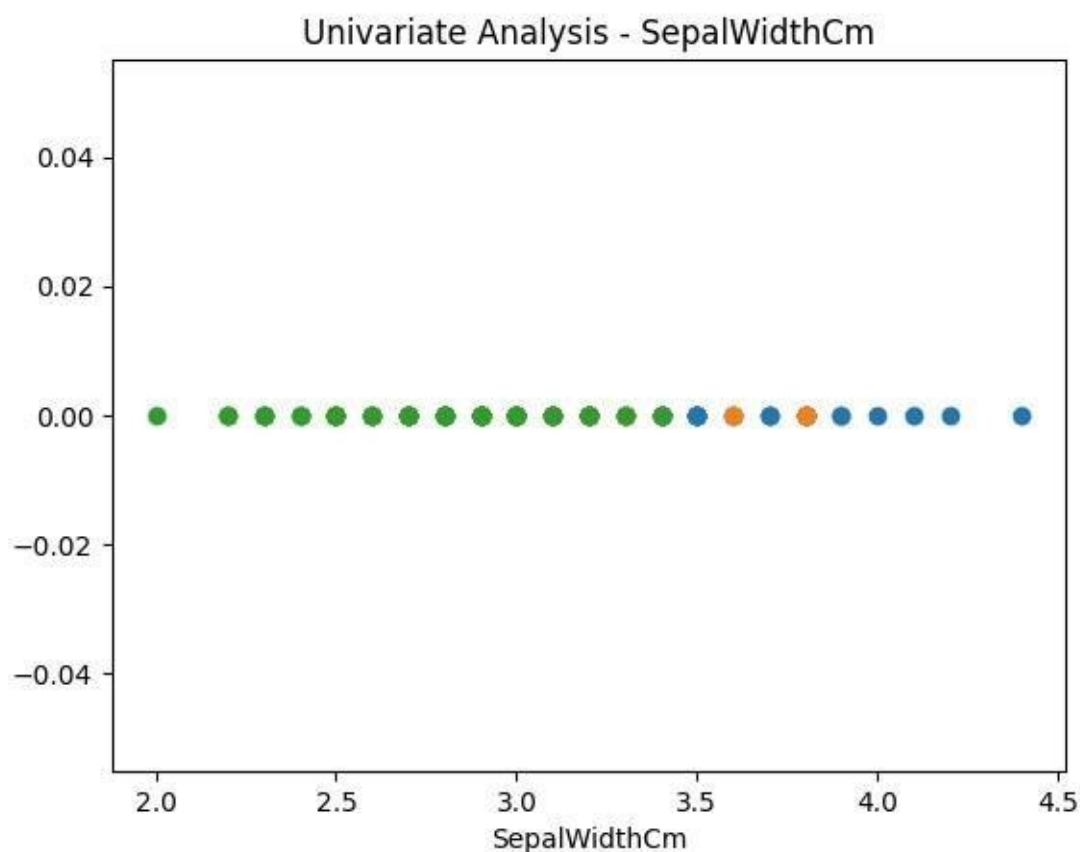
```
#Dataset Shape df.shape
```

```
(150, 6)
```

```
#Separate Species
df_setosa = df[df['Species'] == 'Iris-setosa']
df_virginica = df[df['Species'] == 'Iris-virginica']
df_versicolor = df[df['Species'] == 'Iris-versicolor']
```

#Univariate (SepalWidthCm)

```
plt.scatter(df_setosa['SepalWidthCm'],
np.zeros_like(df_setosa['SepalWidthCm']))
plt.scatter(df_virginica['SepalWidthCm'],
np.zeros_like(df_virginica['SepalWidthCm']))
plt.scatter(df_versicolor['SepalWidthCm'],
np.zeros_like(df_versicolor['SepalWidthCm']))
) plt.xlabel("SepalWidthCm") plt.title("Univariate
Analysis - SepalWidthCm") plt.show()
```



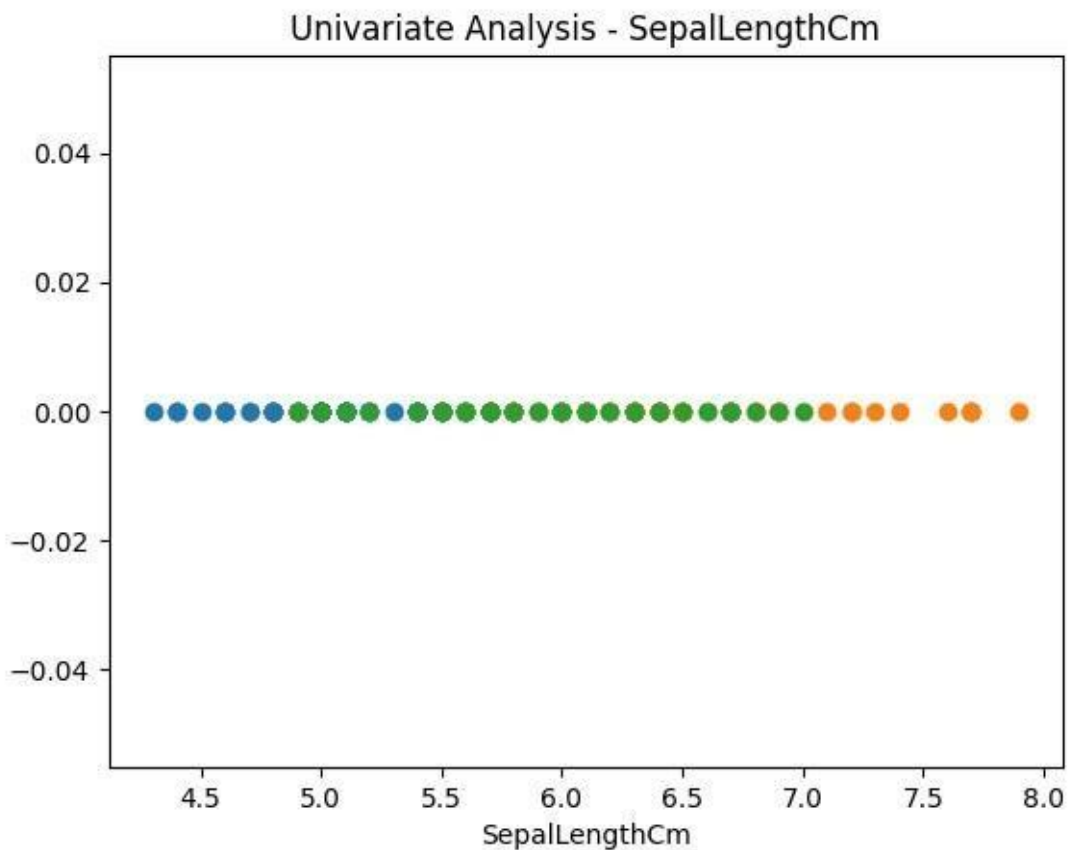
#Univariate (SepalLengthCm)

```
plt.scatter(df_setosa['SepalLengthCm'],
np.zeros_like(df_setosa['SepalLengthCm']))
plt.scatter(df_virginica['SepalLengthCm'],
```

```

np.zeros_like(df_virginica['SepalLengthCm']))
plt.scatter(df_versicolor['SepalLengthCm'],
np.zeros_like(df_versicolor['SepalLengthCm'])
)
plt.xlabel("SepalLengthCm")
plt.title("Univariate Analysis - SepalLengthCm")
plt.show()

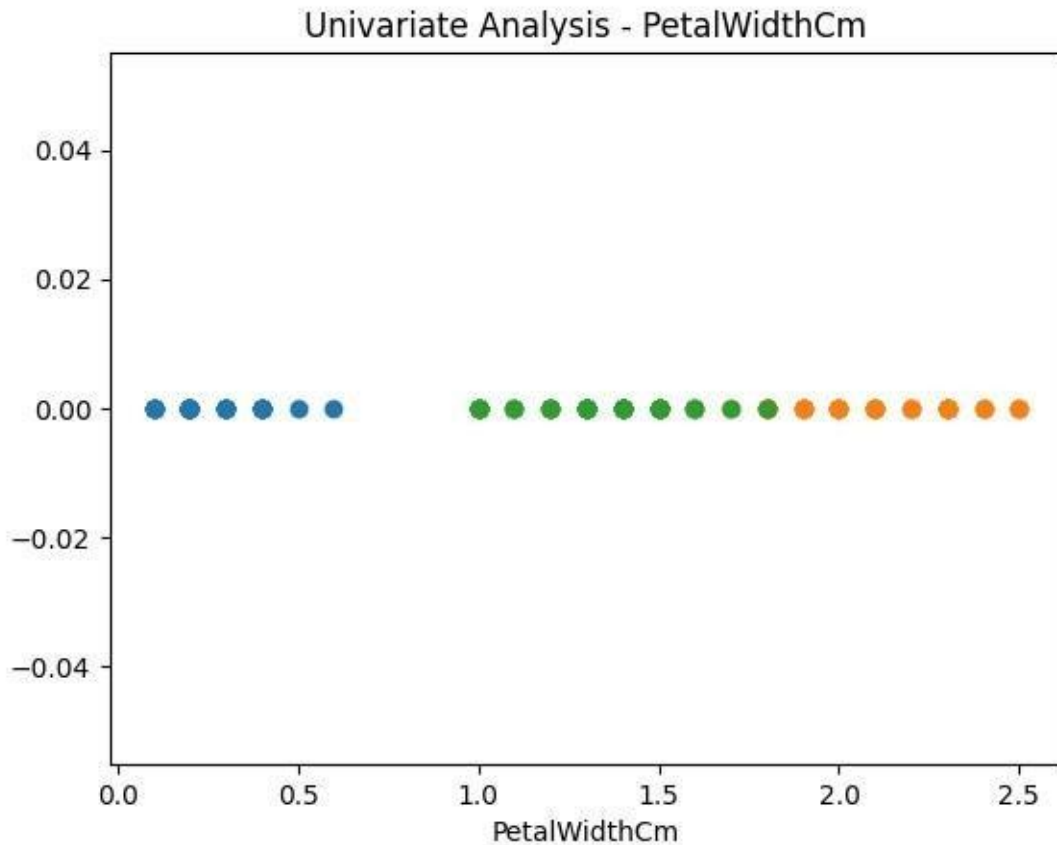
```



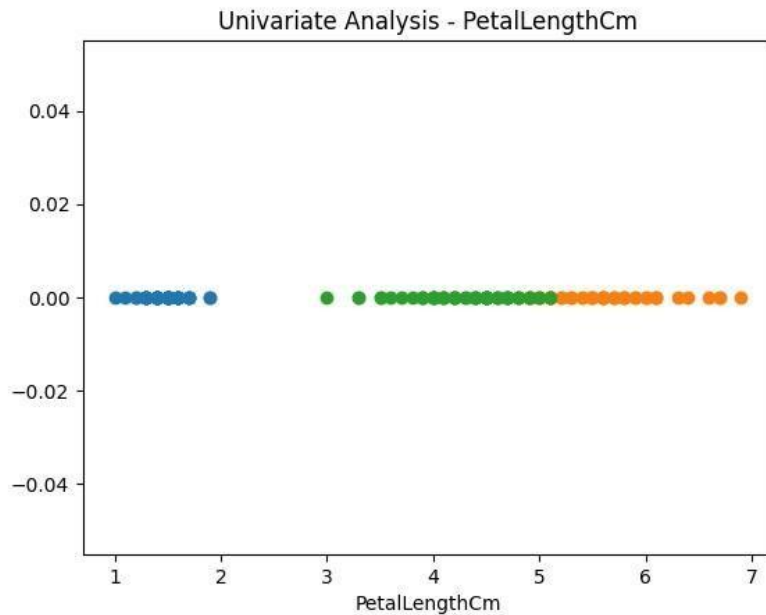
```

#Univariate (PetalWidthCm)
plt.scatter(df_setosa['PetalWidthCm'],
np.zeros_like(df_setosa['PetalWidthCm']))
plt.scatter(df_virginica['PetalWidthCm'],
np.zeros_like(df_virginica['PetalWidthCm']))
plt.scatter(df_versicolor['PetalWidthCm'],
np.zeros_like(df_versicolor['PetalWidthCm']))
plt.xlabel("PetalWidthCm") plt.title("Univariate Analysis
- PetalWidthCm") plt.show()

```



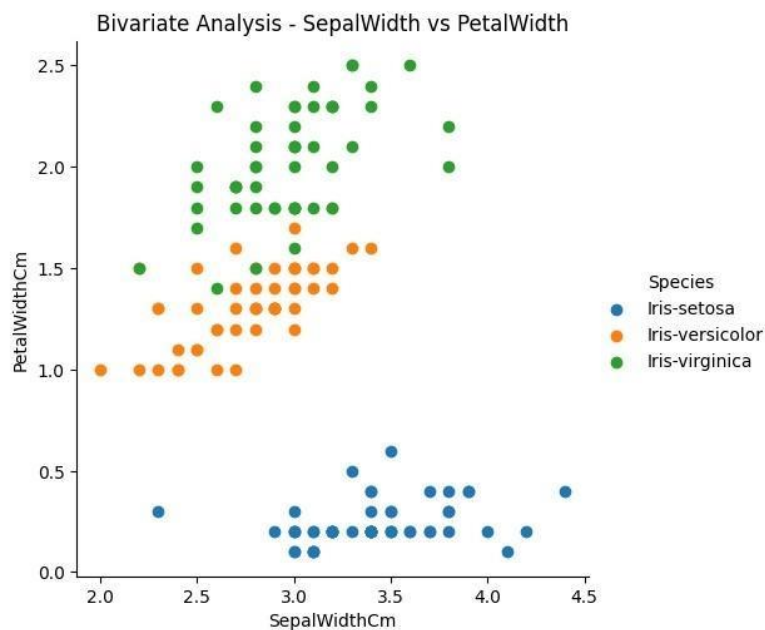
```
#Univariate (PetalLengthCm)
plt.scatter(df_setosa['PetalLengthCm'],
np.zeros_like(df_setosa['PetalLengthCm']))
plt.scatter(df_virginica['PetalLengthCm'],
np.zeros_like(df_virginica['PetalLengthCm']))
plt.scatter(df_versicolor['PetalLengthCm'],
np.zeros_like(df_versicolor['PetalLengthCm']))
)
plt.xlabel("PetalLengthCm")
plt.title("Univariate Analysis - PetalLengthCm")
plt.show()
```

#Bivariate Analysis

#SepalWidth vs PetalWidth

```
sns.FacetGrid(df, hue="Species", height=5) \
    .map(plt.scatter, "SepalWidthCm", "PetalWidthCm") \
    .add_legend()
plt.title("Bivariate Analysis - SepalWidth vs
PetalWidth") plt.show()
```



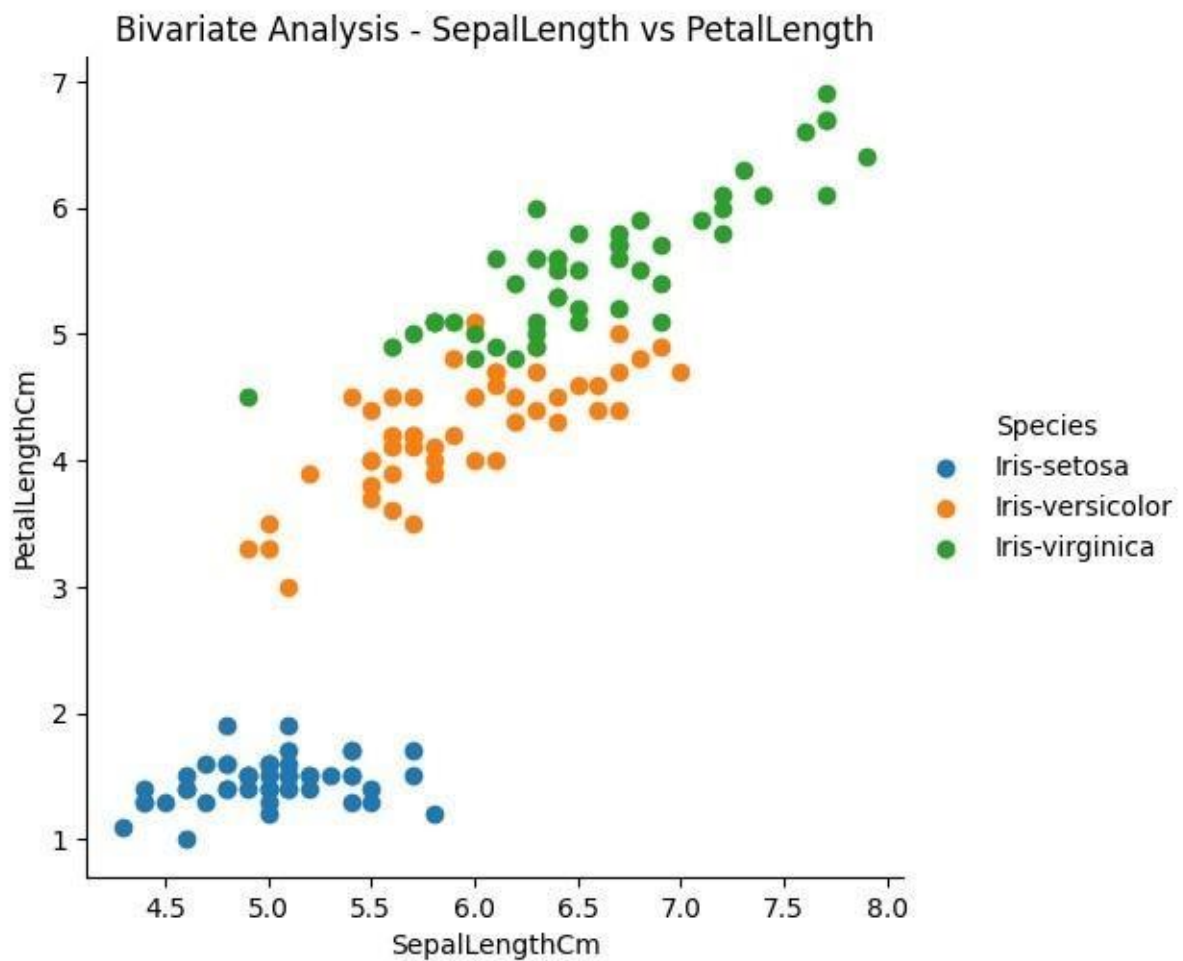
#SepalLength vs PetalLength

```
sns.FacetGrid(df, hue="Species", height=5)
\
```

```

.map(plt.scatter, "SepalLengthCm", "PetalLengthCm") \
.add_legend()
plt.title("Bivariate Analysis - SepalLength vs PetalLength")
plt.show()

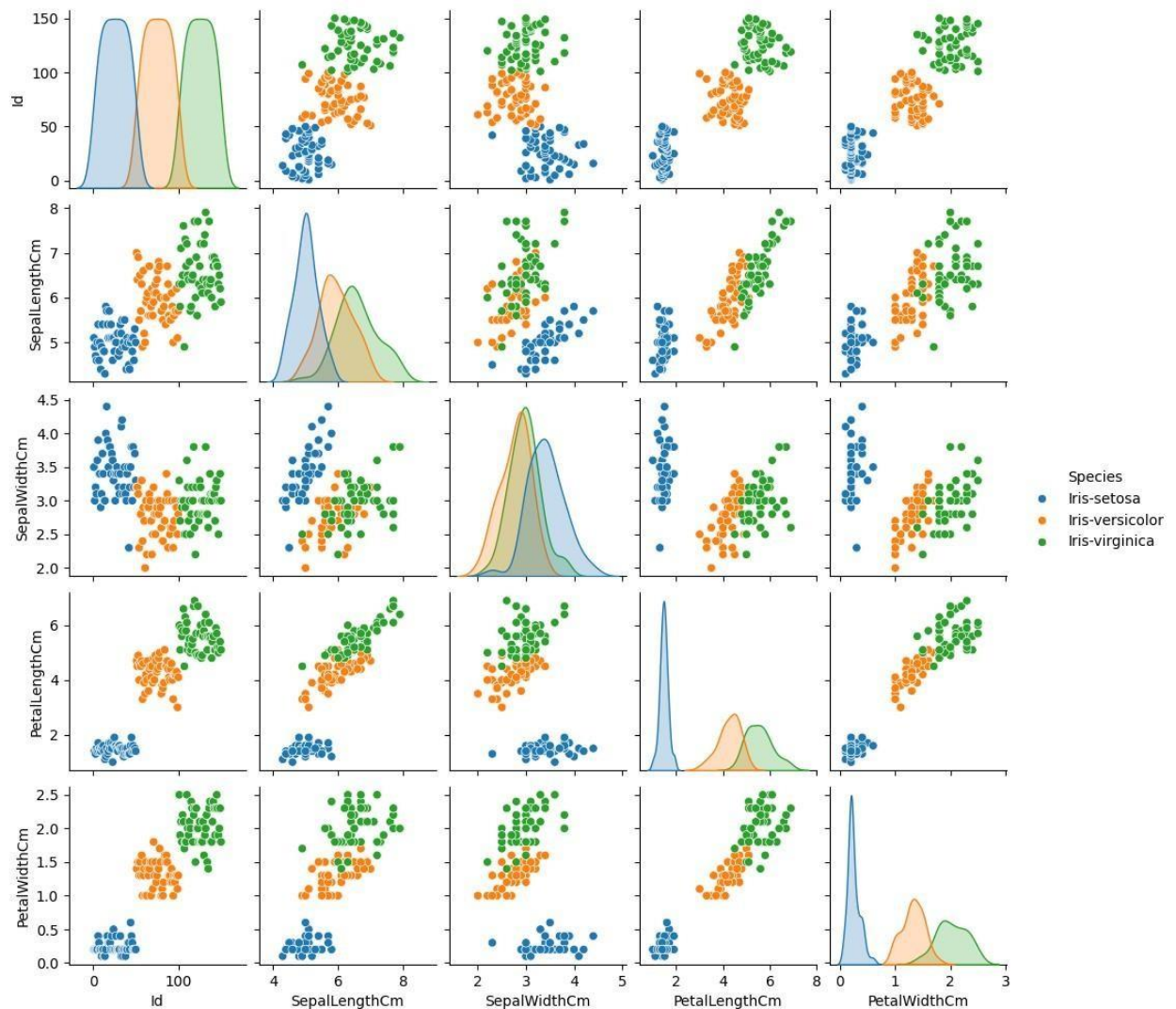
```



```

#Multivariate Analysis
sns.pairplot(df, hue="Species", height=2)
plt.show()

```



Result:

Thus, univariate, bivariate, and multivariate analysis of the Iris dataset was successfully performed using Python. The visualizations clearly show the distribution of features and the relationship between different species of Iris flowers.

Ex. No. : 02

Date:

Register No.: 231701021

Name: Isaitamil S

A PYTHON PROGRAM TO IMPLEMENT SIMPLE LINEAR REGRESSION USING LEAST SQUARE METHOD

Aim:

To implement **Simple Linear Regression** using the **Least Square Method** in Python and analyze the relationship between one independent variable and one dependent variable from the Iris dataset.

Requirements:

Jupyter Notebook with Python, Iris.csv dataset, and required Python libraries.

PROGRAM:

```
import pandas as pd

# Load dataset from your folder df
df = pd.read_csv("iris.csv")

# View first 5 rows
print(df.head()) print(df.info())

print(df.describe())

X = df[['sepal.length', 'sepal.width']] y
= df['petal.length']

from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=42)
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X_train, y_train)
print("Intercept:",
model.intercept_)
print("Coefficients:",
model.coef_)
#Output: Intercept: -
2.365531737511894
Coefficients: [ 1.75575952 -1.34152463]

y_pred = model.predict(X_test)
from sklearn.metrics
import mean_squared_error, r2_score

print("Mean Squared Error:", mean_squared_error(y_test, y_pred))
print("R2
Score:", r2_score(y_test, y_pred))

#Output:

Mean Squared Error: 0.3524875409620241
R2 Score: 0.8924486678535346

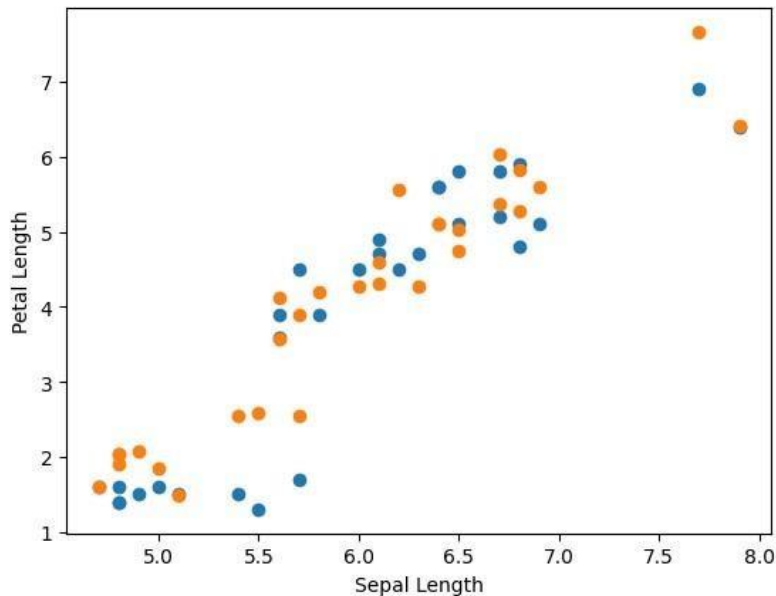
import matplotlib.pyplot as plt

plt.scatter(X_test['sepal.length'], y_test)
```

```
plt.scatter(X_test['sepal.length'], y_pred)

plt.xlabel("Sepal Length") plt.ylabel("Petal
Length") plt.show()
```

#Output:



#Result:

The Simple Linear Regression model was successfully implemented on the Iris dataset. The Least Square Method was used to calculate the best-fit regression line. The model was able to predict the dependent variable with reasonable accuracy, showing a linear relationship between the selected features. The performance was evaluated using metrics such as Mean Squared Error (MSE) and R^2 score.

Ex. No. : 03

Date:

Register No.: 231701021

Name: Isaitamil S

A PYTHON PROGRAM TO IMPLEMENT LOGISTIC MODEL

Aim:

To implement a **Logistic Regression model in Python** to classify **fraudulent and normal credit card transactions**.

Requirements:

Jupyter Notebook with Python, creditcard.csv dataset, and required Python libraries.

PROGRAM:

Step 1: Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

Step 2: Load Dataset `data = pd.read_csv("creditcard.csv")`

```
data.head()
```

	Time	V1	V2	V3	V4	V5	V6	V7
0	0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599
1	0	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.078803
2	1	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	0.791461
3	1	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	1.247203	0.237609
4	2	-1.158233	0.877737	1.548718	0.403034	-0.407193	0.095921	0.592941
5 rows × 9 columns								

Step 3: Dataset Information

```
data.info()
data.describe()
```

```
data.info()
data.describe()

... <class 'pandas.core.frame.DataFrame'>
RangeIndex: 11959 entries, 0 to 11958
Data columns (total 31 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Time        11959 non-null  int64
1   V1          11959 non-null  float64
2   V2          11959 non-null  float64
3   V3          11959 non-null  float64
4   V4          11959 non-null  float64
5   V5          11959 non-null  float64
6   V6          11959 non-null  float64
```

Step 4: Define Features and Target

Target column = **Class**

0 → Normal Transaction 1
→ Fraud Transaction

```
X = data.drop("Class", axis=1) y
= data["Class"]
```

Step 5: Split Training and Testing Data

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Step 6: Train Logistic Regression Model

```
model = LogisticRegression(max_iter=1000)
```

```
model.fit(X_train, y_train)
```

```
model = LogisticRegression(max_iter=1000)

model.fit(X_train, y_train)

... /usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:1145:
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
n_iter_i = check_optimize_result(
    LogisticRegression(max_iter=1000))
```

Step 7: Predict Results y_pred


```
= model.predict(X_test)
```

Step 8: Model Evaluation Accuracy

```
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

```
▶ accuracy = accuracy_score(y_test, y_pred)
  print("Accuracy:", accuracy)

... Accuracy: 0.9991638795986622
```

Confusion Matrix

```
print("Confusion Matrix:") print(confusion_matrix(y_test,
y_pred))
```

```
▶ print("Confusion Matrix:")
  print(confusion_matrix(y_test, y_pred))

... Confusion Matrix:
  [[2377  1]
   [ 1 13]]
```

Classification Report

```
print("Classification Report:")
print(classification_report(y_test, y_pred))
```

```
... Classification Report:
              precision    recall  f1-score   support

     0.0       1.00      1.00      1.00     2378
     1.0       0.93      0.93      0.93        14

 accuracy          0.96
 macro avg         0.96
 weighted avg      1.00
```

Result:

Thus, the **Logistic Regression** model was successfully implemented using **Python** and evaluated using the **Credit Card Fraud Detection** dataset.

Ex. No. : 04

Date:

Register No.: 231701021

Name: Isaitamil S

A PYTHON PROGRAM TO IMPLEMENT SINGLE LAYER PERCEPTRON

Aim:

To implement a **Single Layer Perceptron model in Python** to classify **fraudulent and normal credit card transactions**.

Requirements:

Jupyter Notebook with Python, creditcard.csv dataset, and required Python libraries.

PROGRAM:

Step 1: Import Required Libraries

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.linear_model import Perceptron
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from sklearn.preprocessing import StandardScaler
```

Step 2: Upload Dataset

```
from google.colab import files
uploaded = files.upload()
creditcard.csv
```

Step 3: Load the Dataset

```
data = pd.read_csv("creditcard.csv")
```

```
data.head()
```

...		Time	V1	V2	V3	V4	V5	V6	V7
	0	0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599
	1	0.0	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.078803
	2	1.0	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	0.791461
	3	1.0	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	1.247203	0.237609
	4	2.0	-1.158233	0.877737	1.548718	0.403034	-0.407193	0.095921	0.592941
5 rows × 31 columns									

Step 4: Define Features and Target

Target column:

Class

0 → Normal Transaction 1

→ Fraud Transaction

```
X = data.drop("Class", axis=1) y  
= data["Class"]
```

Step 5: Feature Scaling

Perceptron works better when data is scaled.

```
scaler = StandardScaler()
```

```
X_scaled = scaler.fit_transform(X)
```

Step 6: Split Dataset

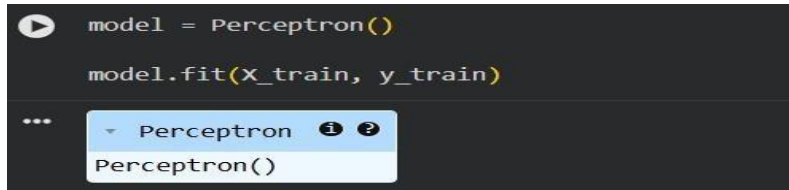
```
X_train, X_test, y_train, y_test = train_test_split(  
    X_scaled, y, test_size=0.2, random_state=42  
)
```

Step 7: Train the Perceptron Model

```
model = Perceptron()
```

```
model.fit(X_train, y_train)
```

```
model = Perceptron()
model.fit(X_train, y_train)
```



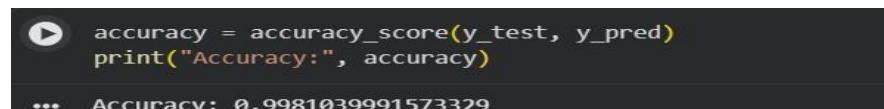
Step 8: Predict the Results

```
y_pred = model.predict(X_test)
```

Step 9: Evaluate the Model Accuracy

```
accuracy = accuracy_score(y_test, y_pred) print("Accuracy:", accuracy)
```

```
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```



Confusion Matrix

```
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

```
Confusion Matrix:
[[56776  88]
 [  20   78]]
```

Classification Report

```
print("Classification Report:")
print(classification_report(y_test, y_pred))
```

```
Classification Report:
              precision    recall  f1-score   support

     0       1.00      1.00      1.00     56864
     1       0.47      0.80      0.59         98

 accuracy          0.9981
 macro avg          0.7350
 weighted avg          0.9981
```

Result:

Thus, the **Single Layer Perceptron** model was successfully implemented using Python and evaluated using the Credit Card Fraud Detection dataset.

Ex. No. : 05

Date:

Register No.: 231701021

Name: Isaitamil S

A Program To Implement Multi Layer Perceptron With Back Propagation

Aim:

To implement a **Multi Layer Perceptron with Backpropagation** to classify **fraudulent and normal transactions**.

Requirements:

Jupyter Notebook with Python, creditcard.csv dataset, and required Python libraries.

PROGRAM:

Step 1: Import Libraries

```
import pandas as pd
import numpy as np
```

```
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

Step 2: Load Dataset

```
data = pd.read_csv("creditcard.csv")

data.head()
```

	Time	V1	V2	V3	V4	V5	V6	V7
0	0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599
1	0	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.078803
2	1	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	0.791461
3	1	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	1.247203	0.237609
4	2	-1.158233	0.877737	1.548718	0.403034	-0.407193	0.095921	0.592941

5 rows × 31 columns

Step 3: Define Features and Target

```
X = data.drop("Class",axis=1) y
= data["Class"]
```

Step 4: Feature Scaling

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Step 5: Split Dataset

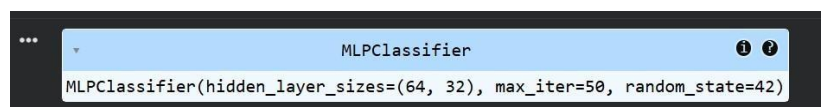
```
X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0.2, random_state=42
)
```

Step 6: Train MLP Model (Backpropagation happens internally)

```
model = MLPClassifier( hidden_layer_sizes=(64, 32), # two hidden
layers
activation='relu',
solver='adam', max_iter=50, random_state=42

)
```

```
model.fit(X_train, y_train)
```



Step 7: Predict Results

```
y_pred = model.predict(X_test)
```

Step 8: Model Evaluation Accuracy

```
accuracy = accuracy_score(y_test, y_pred) print("Accuracy:", accuracy)
```

```
Accuracy: 0.999420666409185
```

Confusion Matrix

```
print("Confusion Matrix:") print(confusion_matrix(y_test, y_pred))
```

```
*** Confusion Matrix:
[[56855   9]
 [   24  74]]
```

Classification Report

```
print("Classification Report:")
print(classification_report(y_test, y_pred))
```

```
*** Classification Report:
              precision    recall  f1-score   support

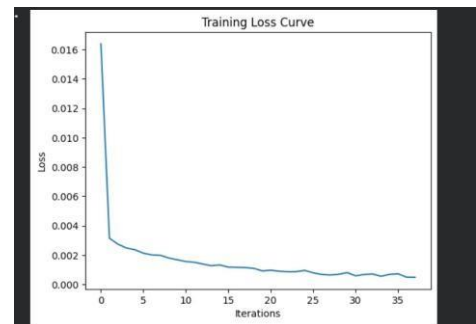
     0       1.00        1.00        1.00    56864
     1       0.89        0.76        0.82        98

 accuracy          0.95        0.88        0.91    56962
 macro avg          0.95        0.88        0.91    56962
 weighted avg          1.00        1.00        1.00    56962
```

Loss Curve

```
import matplotlib.pyplot as plt

plt.plot(model.loss_curve_) plt.title("Training Loss
Curve") plt.xlabel("Iterations") plt.ylabel("Loss")
plt.show()
```



Result:

Thus, the Multi Layer Perceptron with Backpropagation was successfully implemented and evaluated using the dataset.

Ex. No. : 06

Date:

Register No.: 231701021

Name: Isaitamil S

A Python Program To Do Face Recognition Using SVM Classifier

Aim:

To implement **face recognition using Support Vector Machine (SVM).**

Requirements:

Jupyter Notebook with Python, creditcard.csv dataset, and required Python libraries.

PROGRAM:

Step 1: Import Libraries

```
import matplotlib.pyplot as plt

from sklearn.datasets import fetch_lfw_people from
sklearn.model_selection import train_test_split
from sklearn.decomposition import PCA from sklearn.svm import SVC
from sklearn.metrics import classification_report, confusion_matrix,
ConfusionMatrixDisplay
```

Step 2: Load Dataset

```
faces = fetch_lfw_people(min_faces_per_person=50,
resize=0.4)
```

```
X = faces.data y = faces.target
target_names = faces.target_names
```

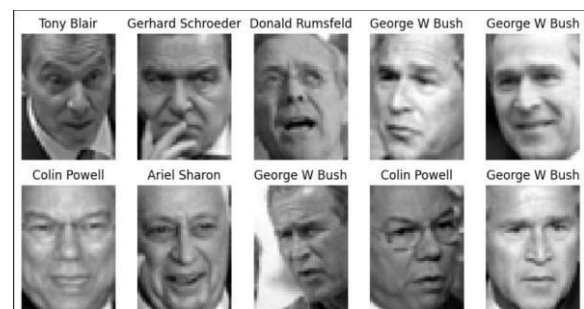
```
print("Dataset shape:", X.shape) print("Number
of classes:", len(target_names))
```

Step 3: Visualize Sample Faces

```
fig, axes = plt.subplots(2, 5, figsize=(10, 5))
```

```
for i, ax in enumerate(axes.flat):
    ax.imshow(faces.images[i], cmap='gray')
    ax.set_title(target_names[y[i]])
    ax.axis('off')
```

```
plt.show()
```



Step 4: Split Dataset


```
X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0.25,
random_state=42)
```

Step 5: Apply PCA (Dimensionality Reduction)

```
pca = PCA(n_components=100, whiten=True, random_state=42)
```

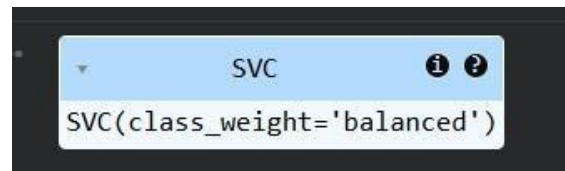
```
X_train_pca = pca.fit_transform(X_train)
```

```
X_test_pca = pca.transform(X_test)
```

Step 6: Train SVM Classifier

```
model = SVC(kernel='rbf', class_weight='balanced')
```

```
model.fit(X_train_pca, y_train)
```



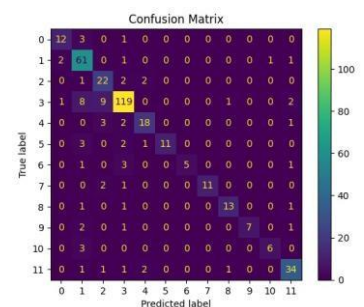
Step 7: Predict Results

```
y_pred = model.predict(X_test_pca)
```

Step 8: Model Evaluation

Confusion Matrix

```
ConfusionMatrixDisplay.from_predictions(y_test, y_pred)
plt.title("Confusion Matrix") plt.show()
```



Classification Report

```
print(classification_report(y_test, y_pred,  
target_names=target_names))
```

	precision	recall	f1-score	support
Ariel Sharon	0.80	0.75	0.77	16
Colin Powell	0.73	0.92	0.81	66
Donald Rumsfeld	0.59	0.81	0.69	27
George W Bush	0.89	0.85	0.87	140
Gerhard Schroeder	0.78	0.75	0.77	24
Hugo Chavez	1.00	0.65	0.79	17
Jacques Chirac	1.00	0.50	0.67	10
Jean Chretien	1.00	0.79	0.88	14
John Ashcroft	0.87	0.81	0.84	16
Junichiro Koizumi	1.00	0.64	0.78	11
Serena Williams	0.86	0.67	0.75	9
Tony Blair	0.83	0.85	0.84	40
accuracy			0.82	390
macro avg	0.86	0.75	0.79	390
weighted avg	0.84	0.82	0.82	390

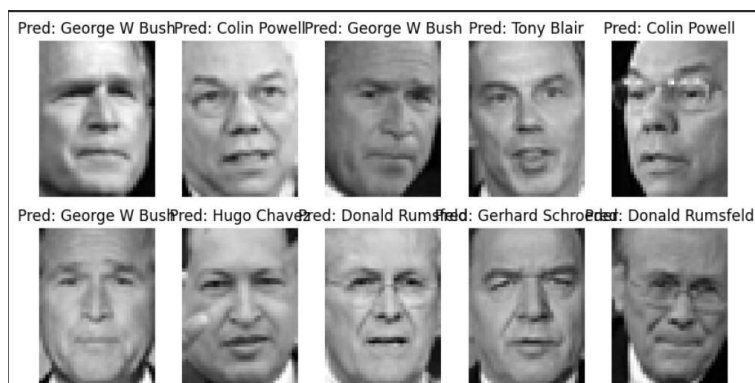
Face Recognition Output:

```
fig, axes = plt.subplots(2, 5, figsize=(10, 5))
```

```
for i, ax in enumerate(axes.flat):
```

```
    ax.imshow(X_test[i].reshape(faces.images[0].shape),  
cmap='gray')    ax.set_title(f'Pred: {target_names[y_pred[i]]}')    ax.axis('off')
```

```
plt.show()
```



Result:

Thus, the Multi Layer Perceptron with Backpropagation was successfully implemented and evaluated using the dataset.

Ex. No. : 07

Date:

Register No.: 231701021

Name: Isaitamil S

A PYTHON PROGRAM TO IMPLEMENT DECISION TREE

Aim:

To implement a **Decision Tree model in Python** to classify **fraudulent and normal credit card transactions**.

Requirements:

Jupyter Notebook with Python, creditcard.csv dataset, and required Python libraries.

PROGRAM:

Step 1: Import Required Libraries

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import matplotlib.pyplot as plt
from sklearn import tree
```

Step 2: Upload Dataset

```
from google.colab import files
uploaded = files.upload()
creditcard.csv
```

Step 3: Load the Dataset

```
data = pd.read_csv("creditcard.csv")
```

```
data.head()
```

...	Time	V1	V2	V3	V4	V5	V6	V7
0	0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599
1	0.0	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.078803
2	1.0	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	0.791461
3	1.0	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	1.247203	0.237609
4	2.0	-1.158233	0.877737	1.548718	0.403034	-0.407193	0.095921	0.592941
5 rows × 31 columns								

Step 4: Define Features and Target

Target column:

Class

0 → Normal Transaction 1

→ Fraud Transaction

```
X = data.drop("Class", axis=1) y  
= data["Class"]
```

Step 5: Train-test split

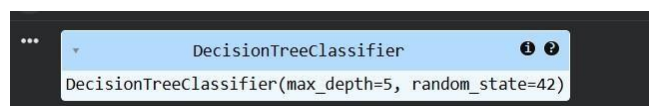
```
X_train, X_test, y_train, y_test = train_test_split(  
X, y, test_size=0.2, random_state=42, stratify=y  
)
```

Step 6: Create Decision Tree model model =

```
DecisionTreeClassifier(max_depth=5, random_state=42)
```

Step 7: Train model

```
model.fit(X_train, y_train)
```



Step 8: Predict the Results

```
y_pred = model.predict(X_test)
```

Step 9: Evaluate the Model Accuracy

```
accuracy = accuracy_score(y_test, y_pred) print("Accuracy:",  
accuracy)
```

```
Accuracy: 0.999553303916762
```

Confusion Matrix

```
print("Confusion Matrix:")  
print(confusion_matrix(y_test, y_pred))
```

```
Confusion Matrix:  
[[91618   9]  
 [   32  126]]
```

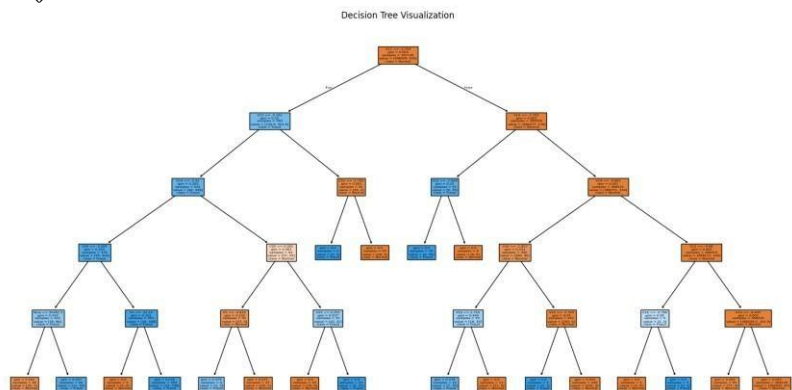
Classification Report

```
print("Classification Report:")  
print(classification_report(y_test, y_pred))
```

```
Classification Report:  
              precision    recall  f1-score   support  
  
   0.0         1.00        1.00        1.00     91627  
   1.0         0.93        0.80        0.86        158  
  
 accuracy          0.97        0.90        0.93     91785  
  macro avg          0.97        0.90        0.93     91785  
 weighted avg          1.00        1.00        1.00     91785
```

Step 10: Visualize Decision Tree

```
plt.figure(figsize=(20,10)) tree.plot_tree(model, filled=True,  
feature_names=X.columns, class_names=['Normal','Fraud']) plt.title("Decision  
Tree Visualization") plt.show()
```



Result:

Thus, the **Decision Tree Classifier** was successfully implemented using **Python** and evaluated using the **Credit Card Fraud Detection** dataset.

Ex. No. : 08

Date:

Register No.: 231701021

Name: Isaitamil S

A PYTHON PROGRAM TO IMPLEMENT BOOSTING

Aim:

To implement a **boosting algorithm (AdaBoost)** on the credit card dataset to detect fraudulent transactions.

Requirements:

Jupyter Notebook with Python, creditcard.csv dataset, and required Python libraries.

PROGRAM:

Step 1: Import Required Libraries

```
import pandas as pd from sklearn.model_selection import  
train_test_split from sklearn.ensemble import  
AdaBoostClassifier from sklearn.tree import  
DecisionTreeClassifier from sklearn.metrics import accuracy_score,  
classification_report, confusion_matrix
```

Step 2: Upload Dataset

```
from google.colab import files  
uploaded = files.upload() Upload:  
  
creditcard.csv
```

Step 3: Load the Dataset

```
data = pd.read_csv("creditcard.csv")  
  
data.head()
```

...	Time	V1	V2	V3	V4	V5	V6	V7
0	0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599
1	0.0	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.078803
2	1.0	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	0.791461
3	1.0	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	1.247203	0.237609
4	2.0	-1.158233	0.877737	1.548718	0.403034	-0.407193	0.095921	0.592941

5 rows × 31 columns

Step 4: Define Features and Target

Target column:

Class

0 → Normal Transaction 1

→ Fraud Transaction

```
X = data.drop("Class", axis=1) y
= data["Class"]
```

Step 5: Train-test split

```
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=42, stratify=y
)
```

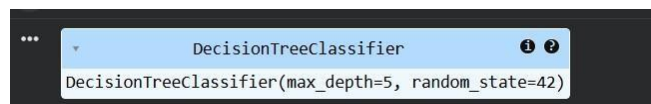
Step 6: Create AdaBoost model base_model =

```
DecisionTreeClassifier(max_depth=1)
```

```
model = AdaBoostClassifier( estimator=base_model, n_estimators=50,
learning_rate=1,random_state=42)
```

Step 7: Train model

```
model.fit(X_train, y_train)
```



Step 8: Predict the Results

```
y_pred = model.predict(X_test)
```

Step 9: Evaluate the Model Accuracy

```
accuracy = accuracy_score(y_test, y_pred) print("Accuracy:",  
accuracy)
```

Confusion Matrix

```
print("Confusion Matrix:")  
print(confusion_matrix(y_test, y_pred))
```

Classification Report

```
print("Classification Report:") print(classification_report(y_test,  
y_pred))
```

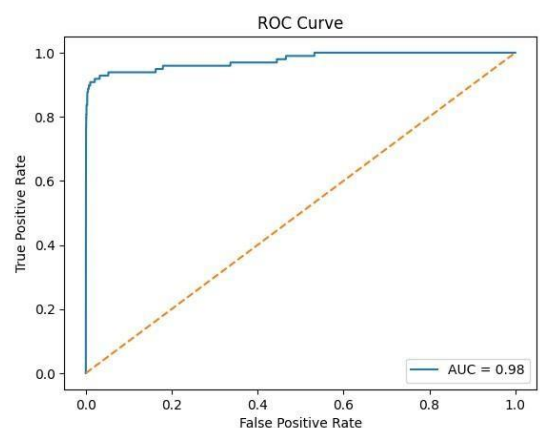
```
AdaBoost Classifier Metrics:  
Accuracy: 0.9990957128071035  
  
Classification Report:  
              precision    recall  f1-score   support  
  
     0.0         1.00      1.00      1.00     91627  
     1.0         0.76      0.70      0.73        158  
  
 accuracy          0.88      0.85      0.86     91785  
 macro avg          1.00      1.00      1.00     91785  
 weighted avg          1.00      1.00      1.00     91785  
  
Confusion Matrix:  
[[91592   35]  
 [   48  110]]
```

Step 9: Evaluate the Model

```
from sklearn.metrics import roc_curve, auc import  
matplotlib.pyplot as plt
```

```
y_prob = model.predict_proba(X_test)[: ,1] fpr, tpr, _ = roc_curve(y_test, y_prob)  
roc_auc = auc(fpr, tpr) plt.plot(fpr, tpr,  
label="AUC = %0.2f" % roc_auc) plt.plot([0,1],  
[0,1], linestyle='--')
```

```
plt.xlabel('False Positive Rate') plt.ylabel('True  
Positive Rate') plt.title('ROC Curve') plt.legend()  
plt.show()
```



Result:

Thus, the boosting model successfully improved classification performance and effectively identified fraudulent transactions.

Ex. No. : 09

Date:

Register No.: 231701021

Name: Isaitamil S

A PYTHON PROGRAM TO IMPLEMENT KNN AND K-MEANS

Aim:

To implement **KNN and K-Means algorithms** on the credit card dataset to perform classification and clustering of transactions.

Requirements:

Jupyter Notebook with Python, creditcard.csv dataset, and required Python libraries.

PROGRAM:

Step 1: Import Required Libraries

```
import pandas as pd from sklearn.model_selection import  
train_test_split from sklearn.neighbors import  
KNeighborsClassifier from sklearn.metrics import  
accuracy_score, classification_report, confusion_matrix from  
sklearn.preprocessing import StandardScaler
```

Step 2: Upload Dataset

```
from google.colab import files  
uploaded = files.upload() Upload:  
creditcard.csv
```

Step 3: Load the Dataset

```
data = pd.read_csv("creditcard.csv")  
  
data.head()
```

...	Time	V1	V2	V3	V4	V5	V6	V7
0	0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599
1	0.0	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.078803
2	1.0	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	0.791461
3	1.0	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	1.247203	0.237609
4	2.0	-1.158233	0.877737	1.548718	0.403034	-0.407193	0.095921	0.592941

5 rows × 31 columns

Step 4: Define Features and Target

```
X = data.drop("Class", axis=1)
y = data["Class"]
```

Step 5: Train-test split

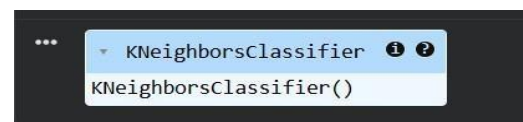
```
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=42, stratify=y
)
```

Step 6: Create KNN model knn =

```
KNeighborsClassifier(n_neighbors=5)
```

Step 7: Train model

```
knn.fit(X_train, y_train)
```



Step 8: Predict the Results

```
y_pred = knn.predict(X_test)
```

Step 9: Evaluate the Model Accuracy

```
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

Confusion Matrix

```
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

Classification Report

```
print("Classification Report:")
print(classification_report(y_test, y_pred))
```

```
... KNN Accuracy: 0.9995435553526912

Classification Report:
              precision    recall  f1-score   support

     0       1.00      1.00      1.00     56864
     1       0.92      0.81      0.86         98

 accuracy      0.96      0.90      0.93     56962
 macro avg      0.96      0.90      0.93     56962
 weighted avg      1.00      1.00      1.00     56962

Confusion Matrix:
[[56857   7]
 [   19   79]]
```

Step 9: Import libraries for K-Means

```
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt
```

Step 10: Apply K-Means (2 clusters: Fraud & Normal)

```
kmeans = KMeans(n_clusters=2, random_state=42)
kmeans.fit(X_scaled)
```

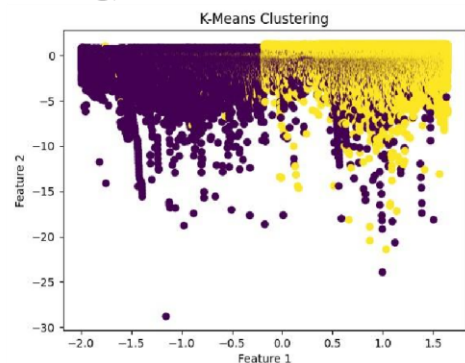
Step 11: Cluster labels and add to dataset

```
clusters = kmeans.labels_
df['Cluster'] = clusters
print(pd.crosstab(df['Class'], df['Cluster']))
```

Cluster	0	1
Class		
0	155038	129277
1	299	193

Step 12: Visualization (using 2 features for plotting)

```
plt.scatter(X_scaled[:,0], X_scaled[:,1], c=clusters)
plt.title("K-Means Clustering")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()
```



Result:

Thus, KNN successfully classified transactions, and K-Means grouped them into clusters, helping identify patterns in fraudulent and normal transactions.

Ex. No. : 10

Date:

Register No.: 231701021

Name: Isaitamil S

A PYTHON PROGRAM TO IMPLEMENT DIMENSIONALITY REDUCTION – PCA

Aim:

To implement **Principal Component Analysis (PCA)** for dimensionality reduction on the credit card dataset.

Requirements:

Jupyter Notebook with Python, creditcard.csv dataset, and required Python libraries.

PROGRAM:

Step 1: Import Required Libraries

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
```

Step 2: Load the Dataset df =

```
pd.read_csv('creditcard.csv')
```

```
print(df.head())
```



...	Time	V1	V2	V3	V4	V5	V6	V7
0	0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599
1	0.0	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.078803
2	1.0	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	0.791461
3	1.0	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	1.247203	0.237609
4	2.0	-1.158233	0.877737	1.548718	0.403034	-0.407193	0.095921	0.592941

5 rows × 31 columns

Step 3: Separate features and target

```
X = df.drop('Class', axis=1)
y = df['Class']
```

Step 4: Feature Scaling

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Step 5: Apply PCA

```
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
```

Step 6: Create DataFrame for visualization

```
pca_df = pd.DataFrame(data=X_pca, columns=['PC1', 'PC2'])
pca_df['Class'] = y
```

Step 7: Explained variance print("Explained Variance Ratio:",

```
pca.explained_variance_ratio_)
```

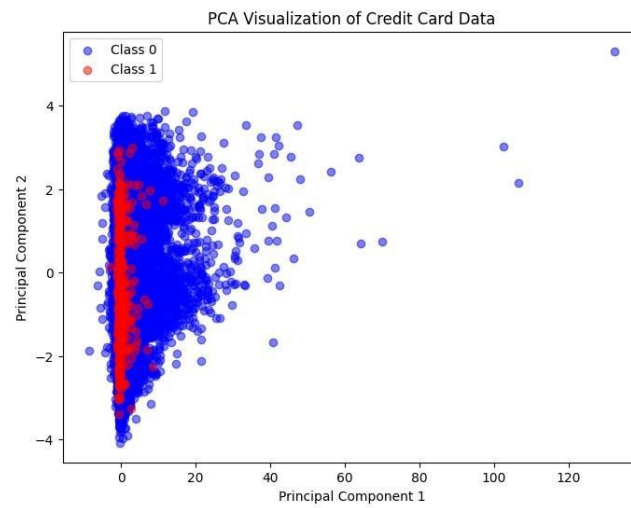
```
... Explained Variance Ratio: [0.06527521 0.05612313]
```

Step 8: Predict the Results

```
plt.figure(figsize=(8,6))
colors = {0: 'blue', 1: 'red'}

for cls in pca_df['Class'].unique():
    subset = pca_df[pca_df['Class'] == cls]
    plt.scatter(subset['PC1'], subset['PC2'],
                c=colors[cls], label=f'Class {cls}', alpha=0.5)

plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.title('PCA Visualization of Credit Card Data')
plt.legend()
plt.show()
```



Result:

Thus, PCA successfully reduced the dataset dimensions while retaining important variance, enabling better visualization and analysis.
